Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни

«Об’єктно-орієнтоване програмування»

на тему

«Розробка багатовіконного інтерфейсу користувача для графічного редактора об’єктів »

Виконала:

Студентка групи ІМ-55

Долгіх Дар’я Михайлівна
Номер у списку групи: 7
Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

Лабораторна робота №5. Розробка багатовіконного інтерфейсу користувача для графічного редактора об’єктів

Мета: Мета роботи – отримати вміння та навички програмувати багатовіконний інтерфейс програми в об’єктно-орієнтованому стилі

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab5.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Коди

Lab5.py

```
import os
import csv
import tkinter as tk
from tkinter import filedialog
from shapes import PointShape, LineShape, RectShape, EllipseShape, LineOOShape,
CubeShape
from my_table import MyTable

class ToolTip:
    def __init__(self, widget, text):
        self.widget = widget
        self.text = text
        self.tip_window = None
        self.widget.bind("<Enter>", self.show_tip)
        self.widget.bind("<Leave>", self.hide_tip)

    def show_tip(self, event=None):
        if self.tip_window or not self.text:
            return
        x = self.widget.winfo_rootx() + 20
        y = self.widget.winfo_rooty() + 25
        self.tip_window = tw = tk.Toplevel(self.widget)
        tw.wm_overrideredirect(True)
```

```
        tw.wm_geometry(f"+{x}+{y}")
        label = tk.Label(tw, text=self.text, background="#ffffe0", relief=tk.SOLID,
borderwidth=1, font=("Tahoma", "8", "normal"))
        label.pack(ipadx=2, ipady=1)

    def hide_tip(self, event=None):
        tw = self.tip_window
        self.tip_window = None
        if tw:
            tw.destroy()

class MyEditor:
    @staticmethod
    def get_instance(root=None):
        if not hasattr(MyEditor.get_instance, "instance"):
            MyEditor.get_instance.instance = MyEditor(root)
        return MyEditor.get_instance.instance

    def __init__(self, root):
        self.root = root
        self.current_tool = "Прямокутник"
        self.shapes = []
        self.max_shapes = 107
        self.current_shape = None

        self.tool_buttons = {}
        self.icons = {}

        self.shape_classes = {
            "Крапка": PointShape,
            "Лінія": LineShape,
            "Прямокутник": RectShape,
            "Еліпс": EllipseShape,
            "Лінія з кружечками": LineOOShape,
            "Куб": CubeShape
        }

        self.table = MyTable(root, on_select=self.on_table_select,
on_delete=self.on_table_delete)

        self.setup_menu()
        self.setup_toolbar()

        self.canvas = tk.Canvas(root, bg="white")
        self.canvas.pack(fill=tk.BOTH, expand=True)
```

```
        self.canvas.bind("<ButtonPress-1>", self.on_press)
        self.canvas.bind("<B1-Motion>", self.on_drag)
        self.canvas.bind("<ButtonRelease-1>", self.on_release)

        self.update_menu_and_title()

    def set_tool(self, tool_name):
        self.current_tool = tool_name
        self.update_menu_and_title()

    def update_menu_and_title(self):
        self.root.title("Лабораторна робота №5 - Багатовіконний інтерфейс
(Singleton Meyers)")
        tools = ["Крапка", "Лінія", "Прямокутник", "Еліпс", "Лінія з кружечками",
"Куб"]
        for idx, tool in enumerate(tools):
            prefix = "✓ " if tool == self.current_tool else "   "
            self.obj_menu.entryconfig(idx, label=f"{prefix}{tool}")

        for name, btn in self.tool_buttons.items():
            if name == self.current_tool:
                btn.config(relief=tk.SUNKEN, bg="#d0d0d0")
            else:
                btn.config(relief=tk.RAISED, bg="#f0f0f0")

    def setup_menu(self):
        menubar = tk.Menu(self.root)
        self.root.config(menu=menubar)

        file_menu = tk.Menu(menubar, tearoff=0)
        file_menu.add_command(label="Зберегти", command=self.save_to_file)
        file_menu.add_command(label="Відкрити", command=self.load_from_file)
        menubar.add_cascade(label="Файл", menu=file_menu)

        self.obj_menu = tk.Menu(menubar, tearoff=0)
        tools = ["Крапка", "Лінія", "Прямокутник", "Еліпс", "Лінія з кружечками",
"Куб"]
        for tool in tools:
            self.obj_menu.add_command(label=tool, command=lambda t=tool:
self.set_tool(t))
        menubar.add_cascade(label="Об'єкти", menu=self.obj_menu)

        table_menu = tk.Menu(menubar, tearoff=0)
        table_menu.add_command(label="Показати / Сховати таблицю",
command=self.table.toggle)
        menubar.add_cascade(label="Таблиця", menu=table_menu)
```

```
        help_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Довідка", menu=help_menu)

    def setup_toolbar(self):
        toolbar = tk.Frame(self.root, bd=1, relief=tk.RAISED, bg="#f0f0f0")
        tools = [
            ("Крапка", "Малювання крапки", "point.png"),
            ("Лінія", "Малювання лінії", "line.png"),
            ("Прямокутник", "Малювання прямокутника", "rect.png"),
            ("Еліпс", "Малювання еліпса", "ellipse.png"),
            ("Лінія з кружечками", "Малювання лінії з кружечками", "line_oo.png"),
            ("Куб", "Малювання каркасу куба", "cube.png")
        ]

        current_dir = os.path.dirname(os.path.abspath(__file__))
        for name, hint, file_name in tools:
            img_path = os.path.join(current_dir, file_name)
            try:
                icon = tk.PhotoImage(file=img_path)
            except tk.TclError:
                icon = tk.PhotoImage(width=24, height=24)

            self.icons[name] = icon
            btn = tk.Button(toolbar, image=icon, command=lambda n=name:
self.set_tool(n), padx=4, pady=4, relief=tk.RAISED)
            btn.pack(side=tk.LEFT, padx=3, pady=2)
            ToolTip(btn, hint)
            self.tool_buttons[name] = btn

        toolbar.pack(side=tk.TOP, fill=tk.X)

    def save_to_file(self):
        filepath = filedialog.asksaveasfilename(defaultextension=".csv",
filetypes=[("CSV", "*.csv")])
        if filepath:
            with open(filepath, "w", newline="", encoding="utf-8") as f:
                writer = csv.writer(f)
                for s in self.shapes:
                    writer.writerow([s.name, s.x1, s.y1, s.x2, s.y2])

    def load_from_file(self):
        filepath = filedialog.askopenfilename(filetypes=[("CSV", "*.csv")])
        if filepath:
            self.shapes.clear()
            with open(filepath, "r", encoding="utf-8") as f:
```

```
                reader = csv.reader(f)
                for row in reader:
                    if len(row) == 5:
                        name, x1, y1, x2, y2 = row
                        shape_class = self.shape_classes.get(name)
                        if shape_class:
                            self.shapes.append(shape_class(int(x1), int(y1),
int(x2), int(y2)))
            self.update_table()
            self.redraw_all()

    def update_table(self):
        data = [(s.name, s.x1, s.y1, s.x2, s.y2) for s in self.shapes]
        self.table.update_data(data)

    def on_table_select(self, index):
        for i, shape in enumerate(self.shapes):
            shape.is_highlighted = (i == index)
        self.redraw_all()

    def on_table_delete(self, index):
        if 0 <= index < len(self.shapes):
            self.shapes.pop(index)
            self.update_table()
            self.redraw_all()

    def redraw_all(self):
        self.canvas.delete("all")
        for shape in self.shapes:
            shape.draw(self.canvas)

    def add_shape(self, shape):
        if len(self.shapes) >= self.max_shapes:
            self.shapes.pop(0)
        self.shapes.append(shape)
        self.update_table()
        self.redraw_all()

    def on_press(self, event):
        for s in self.shapes:
            s.is_highlighted = False
        self.redraw_all()

        shape_class = self.shape_classes.get(self.current_tool)
        if shape_class:
            self.current_shape = shape_class(event.x, event.y, event.x, event.y)
```

```
        if self.current_tool == "Крапка" and self.current_shape:
            self.add_shape(self.current_shape)
            self.current_shape = None

    def on_drag(self, event):
        if self.current_tool == "Крапка":
            self.add_shape(PointShape(event.x, event.y, event.x, event.y))
            return

        if self.current_shape:
            self.current_shape.x2 = event.x
            self.current_shape.y2 = event.y
            self.redraw_all()
            self.current_shape.draw_rubber(self.canvas)

    def on_release(self, event):
        if self.current_tool == "Крапка":
            return

        if self.current_shape:
            self.current_shape.x2 = event.x
            self.current_shape.y2 = event.y
            self.add_shape(self.current_shape)
        self.current_shape = None


if __name__ == "__main__":
    main_window = tk.Tk()
    main_window.geometry("800x600")
    my_editor_global = MyEditor.get_instance(main_window)
    main_window.mainloop()
```

shapes.py

```
class Shape:
    name = ""
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2
        self.is_highlighted = False

    def draw(self, canvas):
        pass
```

```
    def draw_rubber(self, canvas):
        pass

    def draw_highlight(self, canvas):
        if self.is_highlighted:
            x1, x2 = sorted([self.x1, self.x2])
            y1, y2 = sorted([self.y1, self.y2])
            canvas.create_rectangle(x1-5, y1-5, x2+5, y2+5, outline="red", dash=(4,
4), width=2)


class PointShape(Shape):
    name = "Крапка"

    def draw(self, canvas):
        r = 4
        canvas.create_oval(
            self.x2 - r, self.y2 - r,
            self.x2 + r, self.y2 + r,
            fill="black", outline="black"
        )
        self.draw_highlight(canvas)

    def draw_rubber(self, canvas):
        self.draw(canvas)


class LineShape(Shape):
    name = "Лінія"

    def draw(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2, fill="black")
        self.draw_highlight(canvas)

    def draw_rubber(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2, dash=(4, 4),
fill="black")


class RectShape(Shape):
    name = "Прямокутник"

    def draw(self, canvas):
        canvas.create_rectangle(
            self.x1, self.y1, self.x2, self.y2,
            outline="black", fill=""
```

```
        )
        self.draw_highlight(canvas)

    def draw_rubber(self, canvas):
        canvas.create_rectangle(self.x1, self.y1, self.x2, self.y2, dash=(4, 4),
outline="black")


class EllipseShape(Shape):
    name = "Еліпс"

    def draw(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        canvas.create_oval(
            self.x1 - dx, self.y1 - dy,
            self.x1 + dx, self.y1 + dy,
            outline="black", fill="lightgreen"
        )
        self.draw_highlight(canvas)

    def draw_rubber(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        canvas.create_oval(self.x1 - dx, self.y1 - dy, self.x1 + dx, self.y1 + dy,
dash=(4, 4), outline="black")


class LineOOShape(LineShape, EllipseShape):
    name = "Лінія з кружечками"

    def draw(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        LineShape.draw(self, canvas)

        R = 3
        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + R, orig_y1 + R
        EllipseShape.draw(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + R, orig_y2 + R
        EllipseShape.draw(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2
```

```
        self.draw_highlight(canvas)

    def draw_rubber(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        LineShape.draw_rubber(self, canvas)

        R = 3
        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + R, orig_y1 + R
        EllipseShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + R, orig_y2 + R
        EllipseShape.draw_rubber(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2


class CubeShape(RectShape, LineShape):
    name = "Куб"

    def draw(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        shift_x = int((orig_x2 - orig_x1) * 0.3)
        shift_y = int((orig_y2 - orig_y1) * 0.3)

        self.x1, self.y1 = orig_x1 + shift_x, orig_y1 - shift_y
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        RectShape.draw(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x2, orig_y2
        RectShape.draw(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + shift_x, orig_y1 - shift_y
        LineShape.draw(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        LineShape.draw(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y1
        self.x2, self.y2 = orig_x2 + shift_x, orig_y1 - shift_y
```

```
        LineShape.draw(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y2
        self.x2, self.y2 = orig_x1 + shift_x, orig_y2 - shift_y
        LineShape.draw(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2
        self.draw_highlight(canvas)

    def draw_rubber(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        shift_x = int((orig_x2 - orig_x1) * 0.3)
        shift_y = int((orig_y2 - orig_y1) * 0.3)

        self.x1, self.y1 = orig_x1 + shift_x, orig_y1 - shift_y
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        RectShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x2, orig_y2
        RectShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + shift_x, orig_y1 - shift_y
        LineShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        LineShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y1
        self.x2, self.y2 = orig_x2 + shift_x, orig_y1 - shift_y
        LineShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y2
        self.x2, self.y2 = orig_x1 + shift_x, orig_y2 - shift_y
        LineShape.draw_rubber(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2
```

my_table.py

```
import tkinter as tk
from tkinter import ttk
```

```
class MyTable:
    def __init__(self, parent, on_select=None, on_delete=None):
        self.parent = parent
        self.on_select = on_select
        self.on_delete = on_delete

        self.window = tk.Toplevel(parent)
        self.window.title("Таблиця графічних об'єктів")
        self.window.geometry("420x340")
        self.window.protocol("WM_DELETE_WINDOW", self.hide)
        self.window.withdraw()

        columns = ("name", "x1", "y1", "x2", "y2")
        self.tree = ttk.Treeview(self.window, columns=columns, show="headings")
        self.tree.heading("name", text="Назва")
        self.tree.heading("x1", text="x1")
        self.tree.heading("y1", text="y1")
        self.tree.heading("x2", text="x2")
        self.tree.heading("y2", text="y2")

        self.tree.column("name", width=140)
        self.tree.column("x1", width=50)
        self.tree.column("y1", width=50)
        self.tree.column("x2", width=50)
        self.tree.column("y2", width=50)

        self.tree.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        bottom_frame = tk.Frame(self.window)
        bottom_frame.pack(fill=tk.X, pady=5)

        del_btn = tk.Button(bottom_frame, text="Видалити",
command=self._on_delete_btn)
        del_btn.pack(pady=5)

        self.tree.bind("<<TreeviewSelect>>", self._on_select)

    def show(self):
        self.window.deiconify()

    def hide(self):
        self.window.withdraw()

    def toggle(self):
        if self.window.state() == "normal":
            self.hide()
```

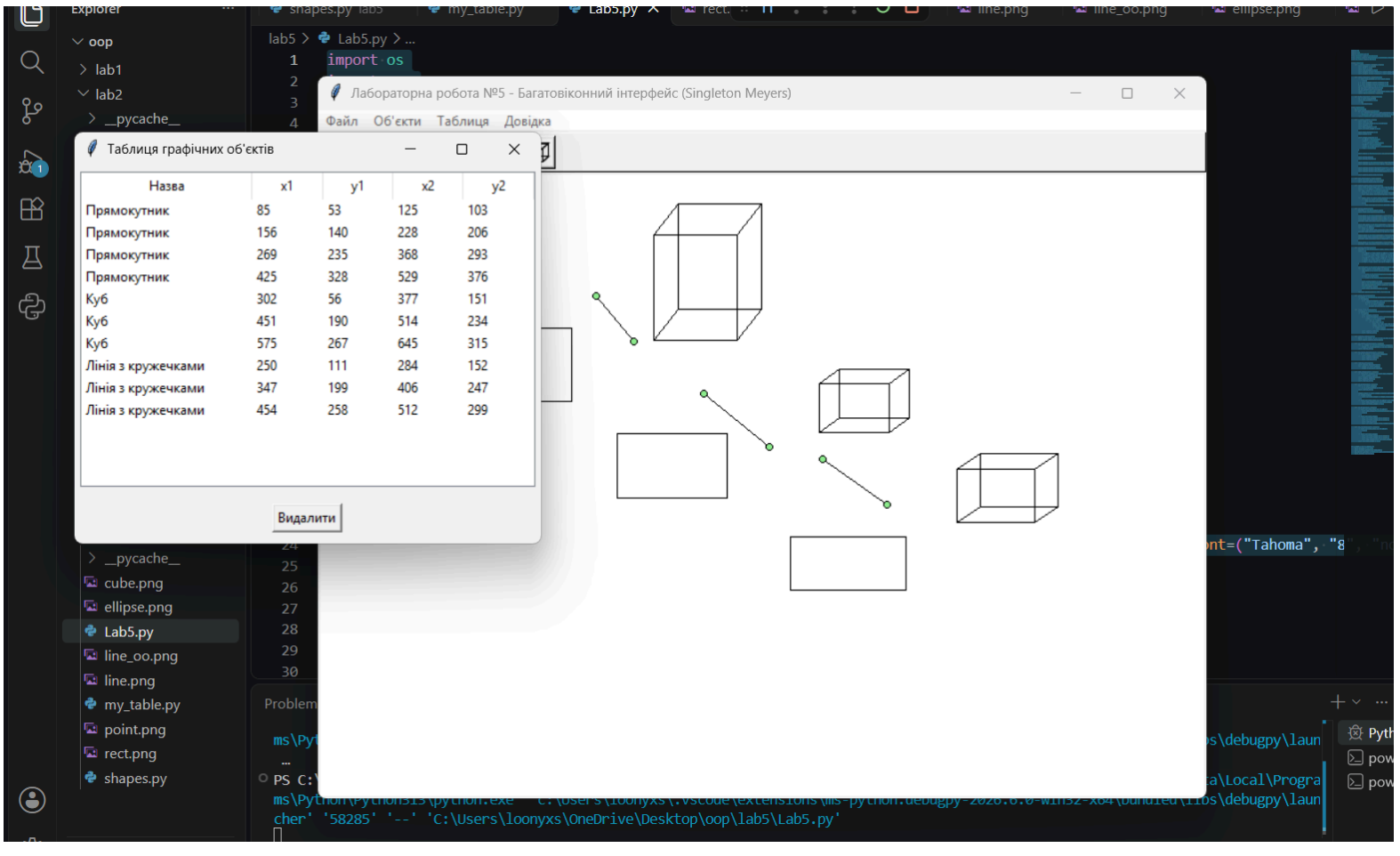
```
        else:
            self.show()

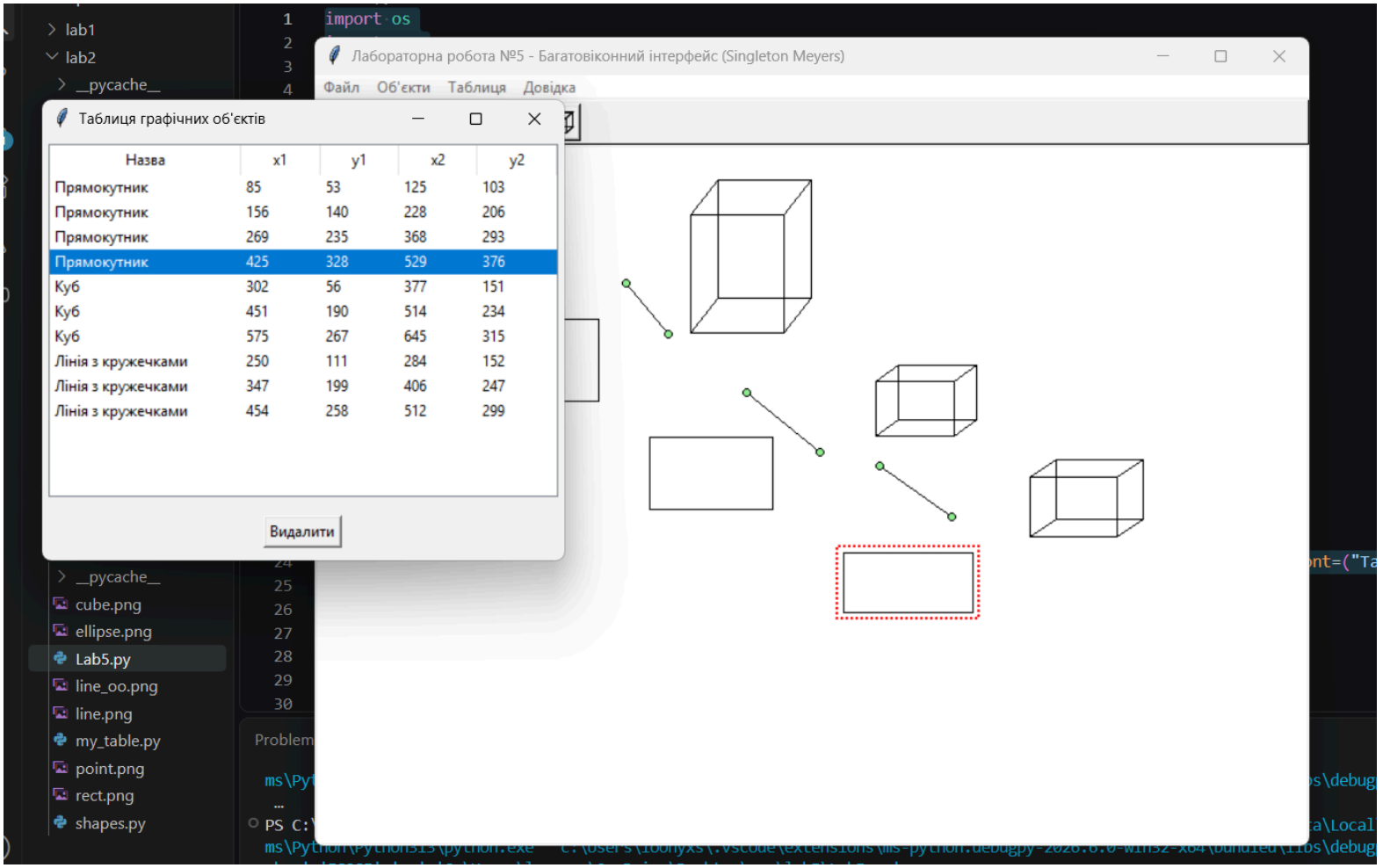
    def update_data(self, data):
        for item in self.tree.get_children():
            self.tree.delete(item)
        for idx, row in enumerate(data):
            self.tree.insert("", tk.END, iid=str(idx), values=row)

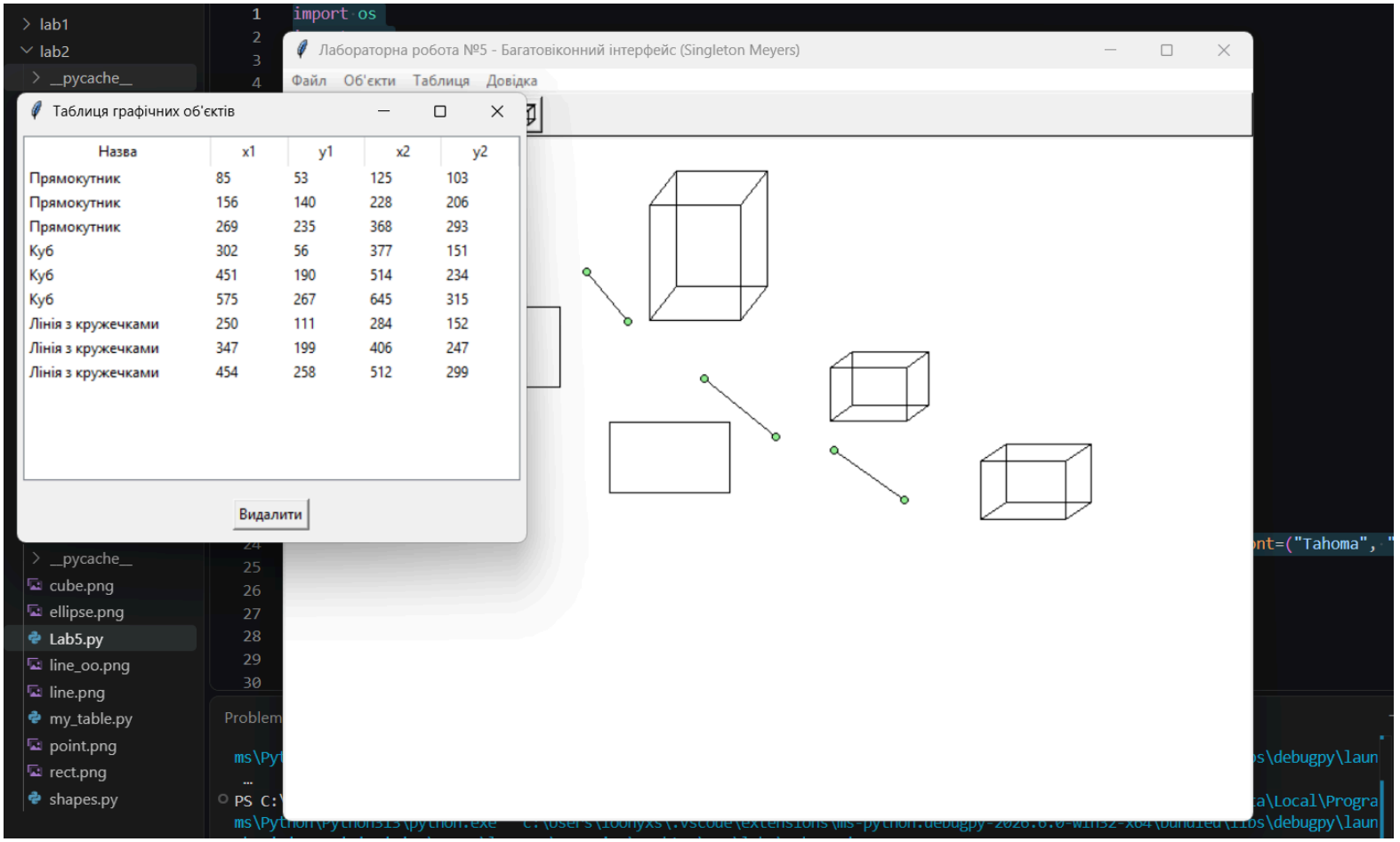
    def _on_select(self, event):
        if self.on_select:
            selected = self.tree.selection()
            if selected:
                self.on_select(int(selected[0]))

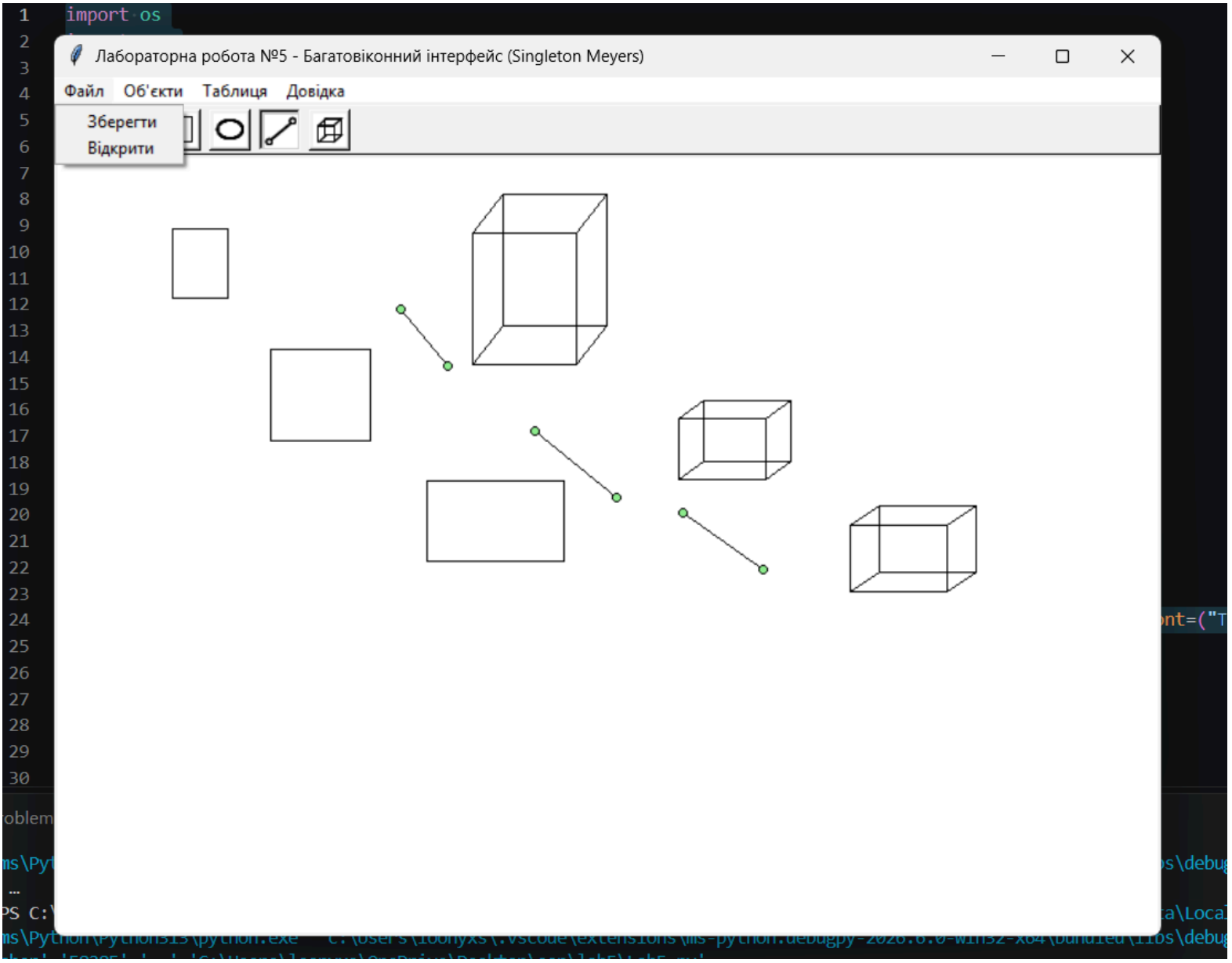
    def _on_delete_btn(self):
        if self.on_delete:
            selected = self.tree.selection()
            if selected:
                self.on_delete(int(selected[0]))
```

Виконання роботи

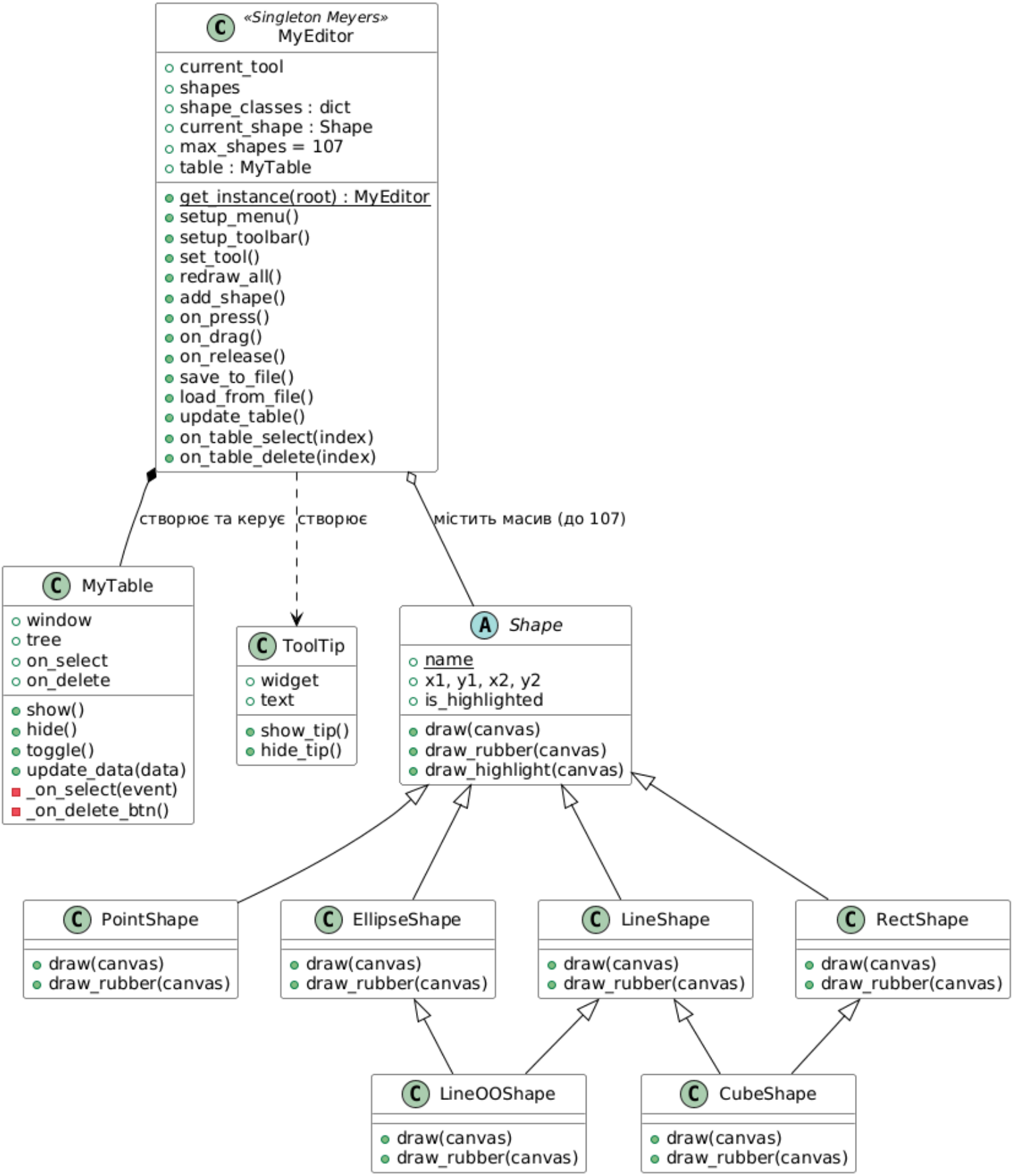



Лабораторна робота №5 - Багатовіконний інтерфейс (Singleton Meyers)
Файл
Об'єкти
Таблиця
Довідка
Таблиця графічних об'єктів
Назва x1 y1 x2 y2
Прямокутник 85 53 125 103
Прямокутник 156 140 228 206
Прямокутник 269 235 368 293
Прямокутник 425 328 529 376
Куб 302 56 377 151
Куб 451 190 514 234
Куб 575 267 645 315
Лінія з кружечками 250 111 284 152
Лінія з кружечками 347 199 406 247
Лінія з кружечками 454 258 512 299
Видалити
lab1
lab2
__pycache__
cube.png
ellipse.png
Lab5.py
line_oo.png
line.png
my_table.py
point.png
rect.png
shapes.py


Лабораторна робота №5 - Багатовіконний інтерфейс (Singleton Meyers)
Файл
Об'єкти
Таблиця
Довідка
Таблиця графічних об'єктів
Назва x1 y1 x2 y2
Прямокутник 85 53 125 103
Прямокутник 156 140 228 206
Прямокутник 269 235 368 293
Куб 302 56 377 151
Куб 451 190 514 234
Куб 575 267 645 315
Лінія з кружечками 250 111 284 152
Лінія з кружечками 347 199 406 247
Лінія з кружечками 454 258 512 299
Видалити
lab1
lab2
__pycache__
cube.png
ellipse.png
Lab5.py
line_oo.png
line.png
my_table.py
point.png
rect.png
shapes.py


import os
Лабораторна робота №5 - Багатовіконний інтерфейс (Singleton Meyers)
Файл
Об'єкти
Таблиця
Довідка
Зберегти
Відкрити

Схема



Висновок

Під час виконання лабораторної роботи №5 я закріпила навички програмування

багатовіконного інтерфейсу та застосування патернів проектування. Відповідно до непарного варіанту завдання, головний об'єкт MyEditor було реалізовано з використанням патерну Meyers Singleton, що гарантує існування лише одного глобального статичного екземпляра редактора протягом роботи програми.

У проект додано новий незалежний модуль my_table, який відповідає за відображення немодального вікна таблиці зі списком намальованих об'єктів. Для забезпечення модульності взаємодія між редактором і таблицею побудована за патерном Callback, що дозволило уникнути прямих залежностей між файлами.

Також успішно реалізовано всі бонусні завдання: виділення фігури на полотні при її виборі в таблиці, зручне видалення об'єкта за допомогою візуальної кнопки в інтерфейсі таблиці з автоматичним оновленням обох вікон, а також збереження і завантаження множини об'єктів у форматі CSV через меню. Мета роботи досягнута, програма відлагоджена та працює коректно.